

Technical Report: On the Redundancy of Task Entities in Object-Centric Event Logs

Waleed Alzamil

January 28, 2026

Abstract

Object-Centric Event Logs (OCEL) provide a standardized format for capturing business processes where multiple object types interact through shared events. OCEL 1.0 [4] defines an event log as a tuple $L = (E, O, \#, \mathcal{R})$ consisting of events, objects, attributes, and event-object relations. OCEL 2.0 [5] extends this with object-to-object (O2O) and event-to-object (E2O) relationships. The dynamic schema approach [3] further introduces temporal O2O relations and E2OAV (event-to-object attribute value) for capturing causality. A Task entity has been proposed as an extension, introducing a separate entity type T with relation $\mathcal{T} \subseteq T \times E$ to group events into tasks.

This technical report demonstrates that the Task entity is **redundant**. We prove: (1) Task-extended logs are isomorphic to standard OCEL logs where tasks are modeled as objects of type *ProcessInstance* (Theorem 1), (2) any query on task-extended logs has an equivalent query on standard OCEL with identical complexity (Theorem 2), and (3) both representations have identical expressiveness (Theorem 3).

We validate these results using two modeling patterns—tasks as objects and tasks as events—both expressible in standard OCEL without requiring a separate Task entity.

Our findings support maintaining the dynamic schema approach [3], recommending tasks be modeled as objects with appropriate type annotations rather than introducing a new entity type.

Keywords: Object-centric process mining, OCEL, event logs, formal methods, task redundancy

1 Introduction

1.1 Context and Motivation

Process mining enables organizations to understand and improve business processes through event data analysis. The field has evolved from traditional case-centric approaches—where each event belongs to exactly one case—to object-centric approaches that recognize business processes involve multiple interacting object types.

The Object-Centric Event Log (OCEL) standard [4] provides a unified format for capturing processes where events may involve any number of objects of different types. Object-centric process mining [9] enables analysis of these complex, interacting processes. The OCEL standard [4] defines an event log as:

$$L = (E, O, \#, \mathcal{R})$$

where E is a set of events, O is a set of objects, $\#$ maps events and objects to their attributes, and $\mathcal{R} \subseteq E \times O$ relates events to participating objects.

The question arises: *should these tasks be represented as a separate entity type in OCEL, or can they be adequately modeled using existing constructs?*

1.2 The Proposed Task Extension

A Task entity has been proposed as an extension to OCEL:

$$L_{task} = (T, E, O, \#, \mathcal{R}, \mathcal{T})$$

where $E, O, \#$ and $\mathcal{R}$ have the same definition as in standard OCEL [4], T is a set of tasks, and $\mathcal{T} \subseteq T \times E$ groups events into tasks. This extension introduces operations: $exec(t)$ (events in task t), $events(o)_t$ (events involving object o within task t), and $trace(o)_t$ (trace of object o within task t).

1.3 Research Questions

This technical report addresses three fundamental questions:

1. **RQ1 (Expressiveness):** Does the Task entity add expressive power to OCEL? That is, can task-extended logs represent information that cannot be captured in standard OCEL?
2. **RQ2 (Query Equivalence):** Are there queries or analyses that can be performed on task-extended logs but not on standard OCEL logs?
3. **RQ3 (Process Mining Value):** Do process mining algorithms benefit from having Tasks as a separate entity type?

1.4 Our Thesis and Contributions

We argue that **the Task entity is redundant** and can be eliminated without loss of information or analytical capability. Tasks can be equivalently represented as:

- Objects (with different approaches for task type granularity, see Section 3.5)
- High-level events that aggregate sub-activities

Both representations are expressible in standard OCEL $L = (E, O, \#, \mathcal{R})$ without requiring a separate Task entity.

Contributions:

1. **Theorem 1 (Information Equivalence):** Task-extended logs L_{task} are **isomorphic** to standard OCEL logs L_{std} where tasks are objects.
2. **Theorem 2 (Query Equivalence):** Any query on task-extended logs translates to an equivalent query on standard OCEL with identical complexity.
3. **Theorem 3 (Expressiveness Equivalence):** Both representations have identical expressive power.
4. **Modeling Patterns:** Two patterns for representing tasks in standard OCEL (tasks as objects, tasks as events).

1.5 Implications

Our findings support:

- Maintaining OCEL as $L = (E, O, \#, \mathcal{R})$ without Task entities
- Conceptual simplicity: fewer fundamental entity types
- Simplified database schemas
- Modeling flexibility: tasks as objects or events based on analytical needs

1.6 Paper Organization

The remainder of this report is organized as follows: Section 2 presents preliminaries on standard OCEL, Section 3 introduces the proposed Task extension as the problem under investigation, Section 4 provides formal analysis establishing Theorems 1-3 and Section 5 Discussion and limitation, Section 6 Conclusion concludes with recommendations.

2 Preliminaries

This section introduces the foundational concepts necessary for understanding our analysis. We begin with the standard OCEL definition, explain object-centric process mining principles, and establish the mathematical notation used throughout this report.

2.1 Object-Centric Event Logs (OCEL)

2.1.1 Motivation for Object-Centric Process Mining

Traditional process mining, as formalized in XES [6], assumes each event belongs to exactly one case. However, real-world processes are more complex: events often involve multiple objects of different types [9]. This simplification leads to well-known issues of *convergence and divergence* [8]. Object-centric process mining [1] addresses these issues by allowing events to relate to any number of objects without assuming a primary case notion.

2.1.2 Formal OCEL Definition

We now present the official OCEL standard definition [4].

Definition 1 (Universes). *Let:*

- $\mathcal{U}_{ev}$ be the universe of events
- $\mathcal{U}_{obj}$ be the universe of objects, where $\mathcal{U}_{ev} \cap \mathcal{U}_{obj} = \emptyset$
- $\mathcal{U}_{act}$ be the universe of activities
- $\mathcal{U}_{time}$ be the universe of timestamps
- $\mathcal{U}_{type}$ be the universe of object types
- $\mathcal{U}_{att} = \{act, time, type, \dots\}$ be the universe of attribute names
- $\mathcal{U}_{val}$ be the universe of attribute values, where $\mathcal{U}_{act} \cup \mathcal{U}_{time} \cup \mathcal{U}_{type} \subseteq \mathcal{U}_{val}$
- $\mathcal{U}_{map} = \mathcal{U}_{att} \rightarrow \mathcal{U}_{val}$ be the universe of partial functions from attributes to values

For any partial function $f \in \mathcal{U}_{map}$, if attribute $a \notin \text{dom}(f)$, we write $f(a) = \perp$ to denote undefined.

Definition 2 (Object-Centric Event Log). *An **object-centric event log** is a tuple:*

$$L = (E, O, \#, \mathcal{R})$$

where:

- $E \subseteq \mathcal{U}_{ev}$ is a finite set of events
- $O \subseteq \mathcal{U}_{obj}$ is a finite set of objects
- $\# \in (E \rightarrow \mathcal{U}_{map}) \cup (O \rightarrow (\mathcal{U}_{time} \rightarrow \mathcal{U}_{map}))$ is an attribute mapping where:
 - For events: $\#(e)$ maps to attributes (e.g., $\#(e)(act)$ is the activity, $\#(e)(time)$ is the timestamp)
 - For objects: $\#(o)(t)$ maps to attributes at time t (allowing time-varying object attributes)
- $\mathcal{R} \subseteq E \times O$ is the event-object relation

Subject to the constraints:

1. $\forall e \in E : \#(e)(act) \in \mathcal{U}_{act}$ (every event has an activity)
2. $\forall e \in E : \#(e)(time) \in \mathcal{U}_{time}$ (every event has a timestamp)
3. $\forall o \in O, \forall t, t' \in \mathcal{U}_{time} : \#(o)(t)(type) = \#(o)(t')(type) \in \mathcal{U}_{type}$ (object type is invariant)

2.1.3 Standard Notation and Operations

For an event log $L = (E, O, \#, \mathcal{R})$, we define:

- $act(L) = \{\#(e)(act) \mid e \in E\}$ is the set of activities in the log
- $types(L) = \{\#(o)(type) \mid o \in O\}$ is the set of object types
- $events(o) = \{e \in E \mid (e, o) \in \mathcal{R}\}$ is the set of events involving object o
- $objects(e) = \{o \in O \mid (e, o) \in \mathcal{R}\}$ is the set of objects involved in event e

Definition 3 (Event Ordering). *Let $L = (E, O, \#, \mathcal{R})$ be an event log. The relation $<_L \subseteq E \times E$ is a total order such that for any $e_1, e_2 \in E$:*

$$e_1 <_L e_2 \implies e_1 \neq e_2 \wedge \#(e_1)(time) \leq \#(e_2)(time)$$

Definition 4 (Object Trace). *For an object $o \in O$, the **event sequence** is:*

$$seq(o) = \langle e_1, e_2, \dots, e_n \rangle$$

where $events(o) = \{e_1, e_2, \dots, e_n\}$ and $e_i <_L e_j$ for all $1 \leq i < j \leq n$.

The **trace** of object o is:

$$trace(o) = \langle \#(e_1)(act), \#(e_2)(act), \dots, \#(e_n)(act) \rangle$$

2.2 Object-Centric Process Mining

Process mining on OCEL includes:

- **Discovery:** Producing process models from event logs
- **Conformance Checking:** Comparing observed behavior against the discovered model
- **Performance Analysis:** Identifying bottlenecks and measuring throughput times

These techniques operate on event sequences, object interactions, and temporal ordering.

In the remainder of this paper, we restrict our definitions to the core OCEL standard [4], excluding extensions from [5] and [3]. Our analysis focuses specifically on entity-based expressiveness. The additions introduced in these works—namely Event-to-Object (E2O) and Object-to-Object (O2O) relationships, time-evolving schemas, and causality (E2OAV)—would increase the complexity of our formalisms without providing additional utility for the proofs that follow.

3 Problem Statement

3.1 Motivation from Practice

3.1.1 Task Management in Business Processes

Organizations use task management systems to coordinate work across teams. These systems track tasks with attributes including: task name, assignee, status, due date, dependencies, and systems involved.

Tasks serve as containers for related activities.

Question: Should tasks be a fundamental entity type, or modeled using existing OCEL constructs?

3.2 The Proposed Task Extension to OCEL

To address task management use cases, a Task entity has been proposed as an extension to the OCEL standard. We now formalize this proposal.

3.2.1 Extended Universes

In addition to the standard OCEL universes (Definition 1), the proposal introduces:

- $\mathcal{U}_{ta}$ - the universe of tasks
- $\mathcal{U}_{tatype}$ - the universe of task types

with the constraints:

- $\mathcal{U}_{ev} \cap \mathcal{U}_{ta} = \emptyset$ (tasks and events are disjoint)
- $\mathcal{U}_{obj} \cap \mathcal{U}_{ta} = \emptyset$ (tasks and objects are disjoint)

The attribute universe is extended: $\mathcal{U}_{att} = \{act, time, type, tatype, \dots\}$

3.2.2 Task-Extended Event Log Definition

Definition 5 (Task-Extended Log). *A **task-extended object-centric event log** is a tuple:*

$$L_{task} = (T, E, O, \#, \mathcal{R}, \mathcal{T})$$

where:

- $T \subseteq \mathcal{U}_{ta}$ is a finite set of tasks
- E, O and $\mathcal{R}$ has exactly the same definition as in standard ocel 2
- $\# \in (T \rightarrow \mathcal{U}_{map}) \cup (E \rightarrow \mathcal{U}_{map}) \cup (O \rightarrow (\mathcal{U}_{time} \rightarrow \mathcal{U}_{map}))$ is an attribute mapping extended to include tasks
- $\mathcal{T} \subseteq T \times E$ is the **task-event relation**

Subject to the constraints:

1. Standard OCEL constraints apply to E, O , and $\mathcal{R}$
2. $\forall t \in T : \#(t)(tatype) \in \mathcal{U}_{tatype}$ (every task has a type)
3. $\forall t \in T : \#(t)(time) \in \mathcal{U}_{time}$ (every task has a timestamp)
4. $\forall (t_i, e_i), (t_j, e_j) \in \mathcal{T} : t_i \neq t_j \implies e_i \neq e_j$ (events belong to at most one task)

3.2.3 Task-Specific Operations

The proposal defines operations specific to task-extended logs:

Definition 6 (Task Operations). *For a task-extended log $L_{task} = (T, E, O, \#, \mathcal{R}, \mathcal{T})$:*

- $tatype(L_{task}) = \{\#(t)(tatype) \mid t \in T\}$ - set of task types
- $exec(t) = \{e \in E \mid (t, e) \in \mathcal{T}\}$ - events contained in task t
- $events(o)_t = \{e \in exec(t) \mid (e, o) \in \mathcal{R}\}$ - events involving object o within task t
- $seq(o)_t = \langle e_1, e_2, \dots, e_n \rangle$ where $events(o)_t = \{e_1, \dots, e_n\}$ and $e_i <_L e_j$ for $i < j$ - event sequence for object o within task t
- $trace(o)_t = \langle \#(e_1)(act), \dots, \#(e_n)(act) \rangle$ where $seq(o)_t = \langle e_1, \dots, e_n \rangle$ - activity trace for object o within task t

3.2.4 Task-Specific Log Projections

The proposal defines projections that extract event logs for specific task types:

Definition 7 (Event Log for Task Type). *For a task-extended log $L_{task} = (T, E, O, \#, \mathcal{R}, \mathcal{T})$ and task type $tat \in \text{tatype}(L_{task})$, the event log for that task type is:*

$$L_{tat} = (E_{tat}, O_{tat}, \#, \mathcal{R}_{tat})$$

where:

- $E_{tat} = \{e \in \text{exec}(t) \mid t \in T, \#(t)(\text{tatype}) = tat\}$
- $O_{tat} = \{o \in O \mid \forall e \in E_{tat} \implies (e, o) \in \mathcal{R}\}$
- $\mathcal{R}_{tat} = \{(e, o) \in \mathcal{R} \mid \forall (e, o) \in E_{tat} \times O_{tat}\}$

3.3 Research Questions

The Task extension raises fundamental questions about the expressiveness and necessity of adding a new entity type to OCEL:

RQ1 (Information Content): Does the task-extended log $L_{task} = (T, E, O, \#, \mathcal{R}, \mathcal{T})$ contain information that cannot be represented in a standard OCEL log $L = (E', O', \#', \mathcal{R}')$?

RQ2 (Query Expressiveness): Are there queries or analytical operations that can be performed on L_{task} but not on standard OCEL?

RQ3 (Process Mining Value): Do process mining algorithms (discovery, conformance checking, performance analysis) benefit from having Tasks as a separate entity type distinct from Objects?

RQ4 (Modeling Flexibility): If Tasks are not a separate entity type, how should task-like concepts be modeled in standard OCEL?

3.4 Our Approach

To answer these research questions, we will:

1. **Prove Information Equivalence (Section 4):** Construct a standard OCEL log L_{std} that is isomorphic to any task-extended log L_{task} , demonstrating that Tasks can be represented as Objects without information loss.
2. **Demonstrate Query Equivalence:** Show that every operation defined on task-extended logs (e.g., $\text{exec}(t)$, $\text{trace}(o)_t$) can be equivalently expressed on standard OCEL with identical computational complexity.
3. **Analyze Process Mining Algorithms:** Examine how major process mining techniques interact with both representations to determine if the Task entity provides algorithmic benefits.

Expected Outcome: We hypothesize that the Task entity is *redundant*, that is, it adds no expressive power to OCEL and can be eliminated in favor of modeling tasks as objects of appropriate types (e.g., *ProcessInstance*, *WorkItem*, *Task*). If this hypothesis is confirmed, it supports maintaining OCEL as defined without the Task extension, ensuring conceptual simplicity while fully supporting task management use cases.

3.5 Assumptions

Our formal analysis relies on the following assumptions, which establish the scope and context for Theorems 1-3.

3.5.1 Modeling Assumption

Assumption 1 (Task-as-Object): We model each task $t \in T$ as an object $o_t \in O$ of type *ProcessInstance*, establishing a bijection $\phi : T \rightarrow O_{proc}$ where $\phi(t) = o_t = t$.

Notes:

- We claim that alternative modeling (tasks as events) yields equivalent conclusions
- Generalization via drill-down operations [7] on task types modeling as object type or attribute produces equivalent models

3.5.2 Task Type Granularity

When modeling tasks as objects, there are two equivalent approaches for handling different task types:

Approach A (Attribute-based): All tasks modeled as single object type with task type as attribute

- $\forall t \in T: \#(\phi(t))(type) = ProcessInstance$
- $\forall t \in T: \#(\phi(t))(task_type) = \#(t)(type)$

Approach B (Type-based): Each task type as distinct object type

- $\forall t \in T: \#(\phi(t))(type) = \#(t)(type)$
- No `task_type` attribute needed

Equivalence: Both approaches preserve information and are equivalent via granularity operations [7]:

- **Drill-down:** Transforms $A \rightarrow B$ by splitting object type based on `task_type` attribute
- **Roll-up:** Transforms $B \rightarrow A$ by merging object types and introducing `task_type` attribute

Schema Impact: With row-based attribute storage [3] and proper indexing (on `type`, composite index on `(object_id, attribute_name)`):

- **Approach A:** Filter *ProcessInstance* objects, then lookup `task_type` attribute via composite index: $O(|O_{PI}|)$ where $|O_{PI}|$ = number of *ProcessInstance* objects
- **Approach B:** Filter directly by specific object type: $O(|O_\tau|)$ where $|O_\tau|$ = number of objects of type τ

Since $|O_\tau| \leq |O_{PI}|$ (specific task type is a subset), Approach B provides better *selectivity*, scanning fewer objects.

Our proofs use Approach A for generality, but both approaches are valid and yield the same conclusions regarding Task redundancy.

3.5.3 Data Structure Assumptions

Assumption 2 (Relational Storage): Event logs are stored in relational databases with proper indexing [3]:

- Index on `objects.type`
- Composite index on `object_attribute_values(object_id, attribute_name)`
- Hash index on `event_object(event_id, object_id)`
- Index on $\mathcal{T}$ by task: $t \mapsto \{e \mid (t, e) \in \mathcal{T}\}$
- Index on $\mathcal{R}$ by object and by event

Scope: Complexity analysis (Theorem 2) assumes relational database operations. This is practical as the hub-and-spoke architecture [3] and most OCEL implementations use relational schemas.

Note: Expressiveness result in Theorems 1 is independent of storage mechanism.

3.5.4 Analysis Scope

Assumption 3 (Functional Equivalence): We focus on *what can be represented* and *what queries can be expressed*

Assumption 4 (Event Partitioning): Following the proposed extension (Definition 5), events belong to at most one task:

$$\forall (t_i, e), (t_j, e) \in \mathcal{T} : t_i = t_j$$

Note: But in the standard OCEL model, events can be related to multiple objects. So for each event we have a constrain on the cardinality of *ProcessInstance* to equal exactly 1, This assumption simplifies analysis without loss of generality.

Our analysis focuses on the core OCEL model [4] and formal expressiveness.

4 Formal Analysis

We prove Task redundancy by constructing an isomorphism between task-extended logs and standard OCEL logs where tasks are modeled as objects.

4.1 Theorem 1: Information Equivalence

Under Assumption 1 (Section 3.5), we model each task $t \in T$ as an object $o_t \in O$ of type *ProcessInstance* via bijection $\phi : T \rightarrow O_{proc}$.

Theorem 1 (Information Equivalence). *For any task-extended log $L_{task} = (T, E, O, \#, \mathcal{R}, \mathcal{T})$, there exists an isomorphic standard OCEL log $L_{std} = (E', O', \#, \mathcal{R}')$ as defined in the OCEL standard [4].*

Proof (Construction):

1. **Event Preservation:** Set $E' = E$ (all events are preserved)
2. **Object Extension:** Define $O' = O \cup O_{proc}$ where:

$$O_{proc} = \{o_t = t \mid t \in T\}$$

That is, $\forall t \in T$, we create a corresponding object o_t of type *ProcessInstance*.

3. **Attribute Mapping (Approach A):** We use the attribute-based approach (see Section 3.5). Define $\#'$ such that:

- $\forall e \in E'$: $\#'(e) = \#(e)$ (event attributes preserved)
- $\forall o \in O$: $\#'(o) = \#(o)$ (original object attributes preserved)
- $\forall o_t \in O_{proc}$:
 - $\#'(o_t)(type) = ProcessInstance$
 - $\#'(o_t)(taskId) = \#(t)(id)$
 - $\#'(o_t)(task_type) = \#(t)(type)$
 - Copy all other task attributes from $\#(t)$ to $\#'(o_t)$

Note: Approach B (type-based) yields an isomorphic result by setting $\#'(o_t)(type) = \#(t)(type)$ directly, omitting the `task_type` attribute. Both approaches preserve all information.

4. **Relation Extension:** Define $\mathcal{R}' = \mathcal{R} \cup \mathcal{R}_{proc}$ where:

$$\mathcal{R}_{proc} = \{(e, o_t) \mid (t, e) \in \mathcal{T} \wedge o_t = t\}$$

That is, if event e belongs to task t in L_{task} , then event e is related to object o_t in L_{std} .

Verification:

Lemma 1 (Bijection). *The mapping $\phi : T \rightarrow O_{proc}$ defined by $\phi(t) = o_t$ is bijective.*

Proof. Injectivity: If $\phi(t_1) = \phi(t_2) \rightarrow o_{t_1} = o_{t_2} \rightarrow t_1 = t_2$ by construction.

Surjectivity: $\forall o \in O_{proc}, \exists! t \in T \mid o = \phi(t)$ by construction. □

Lemma 2 (Information Preservation). *The bijection ϕ preserves all structural information:*

1. $(t, e) \in \mathcal{T} \iff (e, \phi(t)) \in \mathcal{R}'$
2. $exec(t) = events(\phi(t))$
3. $\forall o \in O : events(o)_t = events(o) \cap events(\phi(t))$
4. $\forall a \in \mathcal{U}_{att} : \#(t)(a) = \#(\phi(t))(a)$

Inverse Construction: Given L_{std} with *ProcessInstance* objects, define:

$$\psi : O_{proc} \rightarrow T$$

$$T = \{t = \psi(o) \mid \forall o \in O_{proc}, \#'(o)(type) = ProcessInstance\}$$

$$\mathcal{T} = \{(\psi(o), e) \mid \forall (e, o) \in \mathcal{R}_{proc}, \#'(o)(type) = ProcessInstance\}$$

Since $\psi \circ \phi = id_T$ and $\phi \circ \psi = id_{O_{proc}}$, the representations are isomorphic: $L_{task} \cong L_{std}$. □

4.2 Theorem 2: Query Equivalence

Having established information equivalence, we now prove that all queries on task-extended logs can be equivalently expressed on standard OCEL with identical computational complexity.

Theorem 2 (Query Equivalence). *For any computable query $Q : \mathcal{L}_{task} \rightarrow \mathcal{D}$ on task-extended logs, there exists an equivalent query $Q' : \mathcal{L}_{std} \rightarrow \mathcal{D}$ on standard OCEL logs such that $Q(L_{task}) = Q'(L_{std})$ for all L_{task} and corresponding L_{std} .*

Proof. We establish query equivalence by showing all task-specific operations are expressible in standard OCEL.

Lemma 3 (Cardinality Preservation). *For any task $t \in T$ and corresponding object $\phi(t) \in O_{proc}$:*

$$|\{e \in E \mid (t, e) \in \mathcal{T}\}| = |\{e \in E' \mid (e, \phi(t)) \in \mathcal{R}', \forall t \in T\}|$$

Proof. By construction in Theorem 1, $\mathcal{R}_{proc} = \{(e, o_t) \mid (t, e) \in \mathcal{T} \wedge o_t = t\}$ establishes a one-to-one correspondence between task-event pairs in $\mathcal{T}$ and object-event pairs in $\mathcal{R}_{proc}$. $\forall (t, e) \in \mathcal{T}, \exists! (e, \phi(t)) \in \mathcal{R}_{proc} \subseteq \mathcal{R}'$. Therefore, $|exec(t)| = |events(\phi(t))|$ for all $t \in T$. $\square$

Lemma 4 (Operation Translation). *Let $\phi : T \rightarrow O_{proc}$ be the bijection from Theorem 1. Then:*

1. $exec(t) = \{e \in E \mid (t, e) \in \mathcal{T}\} = \{e \in E' \mid (e, \phi(t)) \in \mathcal{R}'\} = events(\phi(t))$
2. $events(o)_t = \{e \in exec(t) \mid (e, o) \in \mathcal{R}\} = events(o) \cap events(\phi(t))$
3. $trace(o)_t = \langle \#(e)(act) \mid e \in events(o)_t \rangle$ ordered by $\#(e)(time)$

Complexity Analysis:

We analyze the computational complexity of queries on both representations.

Query Q1: Retrieve all events in task/object x

Task-extended log:

- Query: $exec(t) = \{e \in E \mid (t, e) \in \mathcal{T}\}$
- With index on $\mathcal{T}$: $O(1)$ lookup + $O(k)$ retrieval where $k = |exec(t)|$
- Without index: $O(|\mathcal{T}|)$ sequential scan

Standard OCEL:

- Query: $events(\phi(t)) = \{e \in E' \mid (e, \phi(t)) \in \mathcal{R}'\}$
- With index on $\mathcal{R}'$: $O(1)$ lookup + $O(k)$ retrieval where $k = |events(\phi(t))|$
- Without index: $O(|\mathcal{R}'|)$ sequential scan

By Lemma 3, $|exec(t)| = |events(\phi(t))| = k$. Therefore:

- **Indexed access:** Both $O(1 + k)$ - identical
- **Sequential scan:** $O(|\mathcal{T}|)$ vs $O(|\mathcal{R}'|)$ where $|\mathcal{T}| = |\mathcal{R}_{proc}| \leq |\mathcal{R}'|$

Query Q2: Find events involving object o within task t

Task-extended:

$$events(o)_t = \{e \in exec(t) \mid (e, o) \in \mathcal{R}\}$$

Complexity: Retrieve $exec(t)$ in $O(k)$, then filter by checking $(e, o) \in \mathcal{R}$ for each e . Total: $O(k)$ where $k = |exec(t)|$.

Standard OCEL:

$$events(o) \cap events(\phi(t))$$

Complexity: Set intersection $O(\min(|events(o)|, |events(\phi(t))|))$.

Both queries have asymptotically identical computational complexity.

Query Q3: Compute trace of object within task

Task-extended:

$$trace(o)_t = \langle \#(e)(act) \mid e \in events(o)_t \rangle \text{ ordered by } \#(e)(time)$$

Complexity: $O(m)$ to compute $events(o)_t$ plus $O(m \log m)$ to sort, where $m = |events(o)_t|$. Total: $O(m \log m)$.

Standard OCEL: Filter $events(o)$ to include only those in $events(\phi(t))$, extract activities, and sort. Complexity: $O(m)$ for filtering plus $O(m \log m)$ for sorting, where $m = |events(o) \cap events(\phi(t))|$. Total: $O(m \log m)$.

Since $|events(o)_t| = |events(o) \cap events(\phi(t))|$ by Lemma 4, complexity is identical.

Query Q4: Retrieve all events of a specific task type τ

Task-extended log:

$$E_\tau = \{e \in E \mid \exists t \in T : (t, e) \in \mathcal{T} \wedge \#(t)(type) = \tau\}$$

Complexity: $O(|T|)$ to scan tasks plus $O(|\mathcal{T}|)$ to retrieve events = $O(|\mathcal{T}|)$

Standard OCEL (Approach A - Attribute-based):

$$E'_\tau = \{e \in E' \mid \exists o \in O' : (e, o) \in \mathcal{R}' \wedge \#'(o)(type) = ProcessInstance \wedge \#'(o)(task_type) = \tau\}$$

With row-based storage [3] and composite index on (`object_id`, `attribute_name`): Complexity: $O(|O_{PI}|)$ to filter *ProcessInstance* objects and lookup `task.type` via index, plus $O(k)$ for event join where k = matching events

Standard OCEL (Approach B - Type-based):

$$E'_\tau = \{e \in E' \mid \exists o \in O' : (e, o) \in \mathcal{R}' \wedge \#'(o)(type) = \tau\}$$

Complexity: $O(|\mathcal{R}'|)$ - direct type filtering, no attribute join

Analysis: With proper indexing [3], Approach A has complexity $O(|O_{PI}| + k)$ while Approach B has $O(|O_\tau| + k)$. Since $|O_\tau| < |O_{PI}|$ (filtering by specific task type yields a subset of all *ProcessInstance* objects), Approach B provides better *selectivity*.

Key Insight: The bijection $\phi : T \rightarrow O_{proc}$ from Theorem 1 not only preserves information but also preserves **query access patterns**. Since:

1. Each task t maps to exactly one object $\phi(t)$
2. $\mathcal{T}$ and $\mathcal{R}_{proc}$ have identical cardinality per task/object (Lemma 3)
3. Database indices on $\mathcal{T}$ and $\mathcal{R}'$ provide equivalent access paths

all database operations have identical computational complexity. The modeling choice (Task entity vs. *ProcessInstance* object) does not affect query performance. $\square$

4.3 Theorem 3: Expressiveness Equivalence

Having proven information equivalence (Theorem 1) and query equivalence (Theorem 2), we establish that both representations have identical expressive power.

Theorem 3 (Expressiveness Equivalence). *The class of queries expressible on task-extended logs is identical to the class of queries expressible on standard OCEL logs with ProcessInstance objects.*

Proof. By Theorem 1, the representations are isomorphic via bijection ϕ . By Theorem 2, any query on one representation has an equivalent query on the other. Therefore, the query languages have identical expressiveness:

$$\mathcal{Q}(L_{task}) = \mathcal{Q}(L_{std})$$

where $\mathcal{Q}$ denotes the set of expressible queries. $\square$

4.4 Summary of Formal Results

We have established three key theorems:

- **Theorem 1** proves that task-extended logs and standard OCEL logs are isomorphic they contain the same information
- **Theorem 2** proves that any query on task-extended logs can be translated to an equivalent query on standard OCEL with identical complexity
- **Theorem 3** proves that both representations have identical expressive power

Together, these results demonstrate that the Task entity is **formally redundant**: it adds no information, no query capability, and no expressive power to OCEL. The Task extension is therefore unnecessary from a theoretical perspective.

5 Discussion

5.1 Theoretical Implications

Occam’s Razor: The Task entity adds complexity without adding expressive power. The two-entity model (Events and Objects) in OCEL [4] is sufficient.

Generalizability: The same argument applies to other potential extensions (Process, Phase, Stage), all can be modeled as typed objects within the standard OCEL framework [9].

Principle: Domain-specific concepts should be encoded as typed objects, not first-class entities, preserving the generality of the standard.

6 Conclusion

The Task entity proposed as an extension to OCEL is **redundant**.

6.1 Summary of Results

Research Questions Answered:

RQ1 (Expressiveness): Does the Task entity add expressive power?

Answer: No. Theorem 1 establishes $L_{task} \cong L_{std}$ via bijection $\phi : T \rightarrow O_{proc}$.

RQ2 (Query Equivalence): Are there queries expressible only on task-extended logs?

Answer: No. Theorem 2 proves all task queries translate to standard OCEL with identical complexity $O(k)$ or $O(m \log m)$.

RQ3 (Process Mining Value): Do algorithms benefit from Tasks as a separate entity?

Answer: No. Process mining techniques [9] operate identically on both representations, as grouping via $\mathcal{T}$ is equivalent to grouping via $\mathcal{R}$.

6.2 Key Results

1. **Theorem 1:** Task-extended logs isomorphic to standard OCEL with ProcessInstance objects
2. **Theorem 2:** Query equivalence with identical complexity
3. **Theorem 3:** Identical expressive power

6.3 Recommendations

Recommendations:

- **Standards:** Maintain OCEL [5] as $L = (E, O, \#, \mathcal{R})$ without Task extension
- **Practitioners:** Model tasks as objects. For approaches A and B, both are interconvertible via granularity operations [7]. We recommend Approach A as for task management the tasks have the same set of attributes (single table), if there is something exists in a single table, then it is preferable that it corresponds to one object-type or event-type
- **Researchers:** Explore granularity operations [7] and query optimization for task-oriented analysis

6.4 Final Statement

The Task entity is **mathematically proven to be redundant**. It can be completely represented using standard OCEL [5] without loss of information or analytical capability. Adding Tasks would increase complexity without providing benefits.

Therefore, OCEL should remain as $L = (E, O, \#, \mathcal{R})$. Standard OCEL fully supports task management through its existing event and object constructs [9], preserving conceptual information and generality [2, 7].

References

- [1] Business process management workshops: BPM 2023 international workshops, utrecht, the netherlands, september 11–15, 2023, revised selected papers.
- [2] Jan Niklas Adams, Daniel Schuster, Seth Schmitz, Gunther Schuh, and Wil M. P. van der Aalst. Defining cases and variants for object-centric event data. In *2022 4th International Conference on Process Mining (ICPM)*, pages 128–135. IEEE, October 2022.
- [3] Lien Bosmans, Jari Peepkorn, Alexandre Goossens, Giovanni Lugaresi, Johannes De Smedt, and Jochen De Weerd. Dynamic and scalable data preparation for object-centric process mining, October 2024.
- [4] Anahita Farhang Ghahfarokhi, Gyunam Park, Alessandro Berti, and Wil M. P. van der Aalst. Ocel standard. Technical report, RWTH Aachen University, Process and Data Science Group, January 2020. OCEL 1.0 Specification.
- [5] Anahita Farhang Ghahfarokhi, Gyunam Park, Alessandro Berti, and Wil M. P. van der Aalst. Ocel standard (version 2.0), 2021. OCEL 2.0 Specification.
- [6] IEEE Task Force on Process Mining. Xes standard definition, 2016. eXtensible Event Stream standard.
- [7] Shahrzad Khayatbashi, Najmeh Miri, and Amin Jalali. Advancing object-centric process mining with multi-dimensional data operations, November 2024.
- [8] Wil M. P. Van Der Aalst. Object-centric process mining: Dealing with divergence and convergence in event data. In Peter Csaba Ölveczky and Gwen Salaün, editors, *Software Engineering and Formal Methods*, volume 11724, pages 3–25. Springer International Publishing. Series Title: Lecture Notes in Computer Science.
- [9] Wil M. P. van der Aalst. Object-centric process mining: An introduction. In Antonio Cerone, editor, *Formal Methods for an Informal World*, volume 13490 of *Lecture Notes in Computer Science*, pages 73–105. Springer International Publishing, Cham, 2023.

A Mathematical Appendix

This appendix provides detailed mathematical proofs supporting the main theorems presented in this report.

A.1 Appendix A: Formal Bijection Between Representations

Lemma A.1 (Bijective Correspondence): The transformation from L_{task} to L_{std} defined in Theorem 1 establishes a bijection between tasks and ProcessInstance objects.

Proof:

Define the function $\phi : T \rightarrow O_{proc}$ by $\phi(t) = o_t$ where o_t is the ProcessInstance object created for task t .

Injectivity: Suppose $\phi(t_1) = \phi(t_2)$. Then $o_{t_1} = o_{t_2}$. By construction, each task maps to a unique object, so $t_1 = t_2$. Therefore, ϕ is injective.

Surjectivity: $\forall o \in O_{proc} \exists t \in T \mid o = o_t = \phi(t)$ by construction. Therefore, ϕ is surjective.

Since ϕ is both injective and surjective, then it is bijective. This establishes a one-to-one correspondence between tasks and ProcessInstance objects. $\square$

Lemma A.2 (Relation Preservation): The task-event relation $\mathcal{T}$ is completely captured by the extended event-object relation $\mathcal{R}'$.

Proof:

We must show $(t, e) \in \mathcal{T} \iff (e, \phi(t)) \in \mathcal{R}'$.

$(\Rightarrow)$ Suppose $(t, e) \in \mathcal{T}$. By construction of L_{std} , we add (e, o_t) to $\mathcal{R}_{proc}$, and $\mathcal{R}' = \mathcal{R} \cup \mathcal{R}_{proc}$. Therefore, $(e, o_t) = (e, \phi(t)) \in \mathcal{R}'$.

$(\Leftarrow)$ Suppose $(e, \phi(t)) \in \mathcal{R}'$. Then either $(e, \phi(t)) \in \mathcal{R}$ or $(e, \phi(t)) \in \mathcal{R}_{proc}$.

Case 1: If $(e, \phi(t)) \in \mathcal{R}$, this would mean $\phi(t) \in O$ (original objects). But $\phi(t) \in O_{proc}$ by definition, and $O \cap O_{proc} = \emptyset$ by construction. Contradiction.

Case 2: If $(e, \phi(t)) \in \mathcal{R}_{proc}$, then by construction, $(t, e) \in \mathcal{T}$.
Therefore, $(e, \phi(t)) \in \mathcal{R}' \implies (t, e) \in \mathcal{T}$. □

A.2 Appendix B: Information Content Analysis

Lemma B.1 (Attribute Preservation): All task attributes are recoverable from the corresponding ProcessInstance object.

Proof:

For any attribute a and task t :

$$\#(t)(a) = v \iff \#(\phi(t))(a) = v$$

by construction in Theorem 1, where we explicitly copy all task attributes to the corresponding object. Since the copy operation is complete (all attributes transferred) and reversible (original values preserved), information is fully preserved. □

Lemma B.2 (Event Set Equivalence): The set of events contained in a task equals the set of events related to the corresponding object.

Proof:

By definition and Lemma A.2:

$$\begin{aligned} exec(t) &= \{e \in E \mid (t, e) \in \mathcal{T}\} \\ &= \{e \in E \mid (e, \phi(t)) \in \mathcal{R}'\} && \text{(by Lemma A.2)} \\ &= \{e \in E' \mid (e, \phi(t)) \in \mathcal{R}'\} && \text{(since } E' = E\text{)} \\ &= events(\phi(t)) \end{aligned}$$

Therefore, $exec(t) = events(\phi(t))$ for all $t \in T$. □

A.3 Appendix C: Query Translation Examples

Lemma C.1 (Temporal Query Preservation): Temporal queries on task-extended logs can be equivalently expressed on standard OCEL.

Proof by Example:

Query Q1: "Find tasks that completed before deadline"

Task-extended log:

$$\{t \in T \mid \#(t)(completedDate) < \#(t)(dueDate)\}$$

Standard OCEL:

$$\{o \in O' \mid \#'(o)(type) = ProcessInstance \wedge \#'(o)(completedDate) < \#'(o)(dueDate)\}$$

The queries have identical structure. By Lemma A.1, the result sets are in bijection via ϕ . □

Query Q2: "Find objects involved in task t "

Task-extended log:

$$\{o \in O \mid \forall e \in E : (t, e) \in \mathcal{T} \implies (e, o) \in \mathcal{R}\}$$

Standard OCEL:

$$\{o \in O \mid \forall e \in E' : (e, \phi(t)) \in \mathcal{R}' \implies (e, o) \in \mathcal{R}'\}$$

Note: $O \subset O'$ but the query focuses on original objects, not ProcessInstance objects.

By Lemma A.2, $(t, e) \in \mathcal{T} \iff (e, \phi(t)) \in \mathcal{R}'$, so the result sets are identical. □

A.4 Appendix D: Complexity Analysis Details

Lemma D.1 (Space Complexity): The space complexity of L_{std} is $O(|T| + |O| + |E| + |\mathcal{R}| + |\mathcal{T}|)$.

Proof:

Storage requirements for L_{std} :

- Events: $|E'| = |E|$ (unchanged)
- Objects: $|O'| = |O| + |O_{proc}| = |O| + |T|$
- Relations: $|\mathcal{R}'| = |\mathcal{R}| + |\mathcal{R}_{proc}| = |\mathcal{R}| + |\mathcal{T}|$

- Attributes: Same as L_{task} (copied)

Total space: $O(|E| + |O| + |T| + |\mathcal{R}| + |\mathcal{T}|)$.

Compare to L_{task} :

- Tasks: $|T|$
- Events: $|E|$
- Objects: $|O|$
- Relations: $|\mathcal{R}| + |\mathcal{T}|$

Total space: $O(|T| + |E| + |O| + |\mathcal{R}| + |\mathcal{T}|)$.

The space complexity is identical up to constant factors. $\square$

Lemma D.2 (Query Complexity): Queries on L_{std} have the same asymptotic complexity as equivalent queries on L_{task} .

Proof:

Consider the query "Get all events in task/object x ":

Task-extended: Scan $\mathcal{T}$ for pairs (x, e) . Complexity: $O(|\mathcal{T}|)$ worst case, $O(\log |\mathcal{T}| + k)$ with index where k is result size.

Standard OCEL: Scan $\mathcal{R}'$ for pairs (e, x) . Complexity: $O(|\mathcal{R}'|)$ worst case, $O(\log |\mathcal{R}'| + k)$ with index.

Since $|\mathcal{T}| \subseteq |\mathcal{R}'|$ (specifically, $|\mathcal{T}| = |\mathcal{R}_{proc}| \subseteq |\mathcal{R}'|$), and both use standard database indexing, the complexity is asymptotically identical.

Similar analysis applies to other query patterns (join queries, filters, aggregations). $\square$

A.5 Appendix E: Inverse Transformation

Lemma E.1 (Reversibility): Given L_{std} with ProcessInstance objects, we can reconstruct L_{task} .

Proof (Constructive):

Define the inverse transformation $\psi : O_{proc} \rightarrow T$:

For each $o \in O'$ where $\#'(o)(type) = ProcessInstance$:

1. Create task $t_o = \psi(o)$
2. Set $\#(t_o) = \#'(o)$ (copy all attributes)
3. $\mathcal{T} = \{(o, e) \mid \forall (e, o) \in \mathcal{R}' : \#'(o)(type) = ProcessInstance\}$

Set $T = \{t_o = o \mid \forall o \in O', \#'(o)(type) = ProcessInstance\}$.

This reconstructs $L_{task} = (T, E, O, \#, \mathcal{R}, \mathcal{T})$ where:

- $E = E'$ (events unchanged)
- $O = O' \setminus O_{proc}$ (remove ProcessInstance objects)
- $\mathcal{R} = \mathcal{R}' \setminus \mathcal{R}_{proc}$ (remove ProcessInstance relations)
- $\mathcal{T}$ reconstructed as above

By construction, $\psi \circ \phi = id_T$ and $\phi \circ \psi = id_{O_{proc}}$, confirming the transformations are inverses. $\square$

A.6 Appendix F: Generalization to Multiple Task Types

The proofs generalize naturally to scenarios with multiple task types, supporting both Approach A and Approach B (see Section 3.5).

Lemma F.1: For any number of task types, the transformation from L_{task} to L_{std} preserves information regardless of whether Approach A or Approach B is used.

Proof:

Let $TT = \{tt_1, tt_2, \dots, tt_n\}$ be the set of task types in L_{task} .

Approach A (Attribute-based): All tasks map to single object type:

- $\forall t \in T: \#'(\phi(t))(type) = ProcessInstance$
- $\forall t \in T: \#'(\phi(t))(task_type) = \#(t)(type)$

- Bijection $\phi : T \rightarrow O_{proc}$ remains valid for all task types

Approach B (Type-based): Each task type maps to distinct object type:

- For each $tt_i \in TT$: define object type $ot_i = tt_i$
- For each task t with $\#(t)(type) = tt_i$: create o_t with $\#'(o_t)(type) = ot_i$
- Bijection per type: $\phi_i : \{t \in T \mid \#(t)(type) = tt_i\} \rightarrow \{o \in O_{proc} \mid \#'(o)(type) = ot_i\}$
- Overall bijection: $\phi = \bigcup_{i=1}^n \phi_i$

Both approaches preserve all information. All previous lemmas hold for each task type separately, and therefore hold for the entire log. $\square$